

same type and len and constructor ku set(())

Acesing a set item:

for loop use panalam while use panalam(len use pannil)
or indha maari use panalam:

```
Check if "banana" is present in the set:

thisset = {"apple", "banana", "cherry"}

print("banana" in thisset)

Try it Yourself >
```

Example

Check if "banana" is NOT present in the set:

```
thisset = {"apple", "banana", "cherry"}

print("banana" not in thisset)
```

ADDING A SET ITEM:

.add() nu kuduthu item add panalam

rendu set add pananam na setno1.update(setno2)
nu kudukanum

rendu collection add pananam na adhu set ah dhan
irukanum nu avasiyam illa it can be any collection
like set list tuple

Removing A SET ITEM:

.remove() use panalam... or discard use panalam..

```
Remove "banana" by using the .discard() method.

thisset = {"apple", "banana", "cherry"}

thisset.discard("banana")

print(thisset)
```

or .pop() use panalaam for cutting out a random
indexitem and storing it in a variable

```
Remove a random item by using the .pop() method:

thisset = {"apple", "banana", "cherry"}

x = thisset.pop()

print(x)

print(thisset)
```

.clear() use panna set clear aairum

Looping items in set:

```
Loop through the set, and print the values.

thisset = {"apple", "banana", "cherry"}

for x in thisset:
    print(x)
```

Union - Rendu set combine pannum duplicate
remove pannum..

```
Python

a = {"apple", "banana"}
b = {"banana", "cherry"}

c = a.union(b)
print(c)

Output:

Code

{'apple', 'banana', 'cherry'}

duplicate "banana" once dhan varum
```

either .union() and | use panikalam

.update kudutha we can insert items from one set
to another set

.intersection use panna common ah irukura items
matum display pannum or & use panalam for same
purpose

Join sets that contains the values [True](#), [False](#), [1](#),
and [0](#), and see what is considered as duplicates.

Difference (Set)

- difference() method returns a new set
- keeps items from the **first set only**
- removes items that are present in the other set(s)
- original set **will not change**
- operator form: -

Syntax:

set1.difference(set2)

Example:

```
a = {1,2,3}
b = {2,3,4}
print(a - b)
print(a.difference(b))
```

Update version (changes original set):

a.difference_update(b)

Membership (Set)

- used to check whether an item exists in a set
- keyword used: in
- returns True or False
- not in is used to check absence
- membership checking in sets is fast

Unpacking the tuples

```
Unpacking a tuple:

fruits = ("apple", "banana", "cherry")

(green, yellow, red) = fruits

print(green)
print(yellow)
print(red)
```

Using Asterisk*

If the number of variables is less than the number of values, you can add an * to the variable name and the values will be assigned to the variable as a list.

Example

Assign the rest of the values as a list called "rest":

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")

(green, yellow, red) = fruits

print(green)
print(yellow)
print(red)
```

Looping a tuple:

for loop use pannil items print panalam or index
number use pannil panalaam...using for I in
range(len(tuplename)): print(tuplename[i])

Math operations use panalam...:

like adding two tuples using a third tuple and printing it

if we wanna multiply the items same
innorunetuple name potu * number of ttimes potu
panikalam

If Statement:

Python supports the usual logical conditions from mathematics:

- Equals: a == b
- Not Equals: a != b
- Less than: a < b
- Less than or equal to: a <= b
- Greater than: a > b
- Greater than or equal to: a >= b

These conditions can be used in several ways, most commonly in "if statements" and loops.

An "if statement" is written by using the [if](#) keyword
So main ah mithun unake teriyum daa...if
(condition):

Or elif(nested ku) or else(last part) avalo dhan
idhula periya condition nu onnum illa

Short hand if else:

```
a = 2
b = 330
print("A") if a > b else print("B")
```

Summa oru line expression ku ipdi padichrvom..

Innoru shorthand...

```
a = 30
b = 20
bigger = a if a > b else b
print("Bigger is", bigger)
```

Nested If :

```
score = 85
attendance = 90
submitted = True

if score >= 60:
    if attendance >= 80:
        print("Pass with good standing")
    else:
        print("Pass but missing assignment")
    else:
        print("Pass but low attendance")
else:
    print("Fail")
```

Namma and or kooda use panalam...

Pass - null operation -acts as a placeholder
Logic future la eludhanum andha maari edathula
idha use panikalam

Match statement (Switch

statement of Python):

Switch maari acting...so perusa onum difference
illa

Match expression:

Case condition:

Code block

Case condition:

Code block

sAME...

Sample code is:

```
day = 4
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
```

Default value is case _: underscore is very

variable=dictionaryname.get("keyname")

to add a new key in dictionary:

dictionaryname["newkeyname"]="newvaluena
e"

to access keys....dictonaryname.keys()

to access values....dictionary.values()

Changing items in dictionary just
dictionaryname["keyname"]=value;
that's it..

if multiple keys

Use update() when:

- multiple keys update
- merge dictionaries

Python

```
d.update({"age":25, "name":"Mithun"})
```

.pop() - remove random item or a specific key

.del(keyname) - deletes a specific keyname

del dictionaryname - deletes the specific dictionary

for clearing dictname.clear()

looping same dhan for or while

for na mothama podalam or values matum
podalam

```
for x in thisdict.values():
    print(x)
```

for both key and value....

```
for x, y in thisdict.items():
    print(x, y)
```

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
for x in thisdict:
  print(x)
```

to copy dictname.copy()

Nested Dictionaries:

a dictionary can contain dictionaries, this is called
nested dictionaries

Program:

```
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}
```

print(myfamily)

Output:

```
{'child1': {'name': 'Emil', 'year': 2004}, 'child2': {'name': 'Tobias', 'year': 2007}, 'child3': {'name': 'Linus', 'year': 2011}}
```

```
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}

for x, obj in myfamily.items():
    print(x)

    for y in obj:
        print(y + ': ', obj[y])
```

Dictionary Methods:

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key value pair
setdefault()	Returns the value of the specified key. If the key does not exist, insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

print(thisdict["year"])

Try it Yourself >
```

Looping a list:

Basic Syntax:

```
thislist = ["apple", "banana", "cherry"]
for x in thislist:
    print(x)
```

Looping through a length of a list:

```
thislist = ["apple", "banana", "cherry"]
for i in range(len(thislist)):
    print(thislist[i])
```

Using a While loop:

```
thislist = ["apple", "banana", "cherry"]
i = 0
while i < len(thislist):
    print(thislist[i])
    i += 1
```

While Statement:

Oru condition true va irukura varaikum
andha loop execute aagum

```
i = 1
while i < 6:
    print(i)
    i += 1
```

Using break we can break the loop is
condition is true under specific
conditions

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

With the [continue](#) statement we can
stop the current iteration, and continue
with the next:

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

For Loop – Complete Notes (Python)

What is a For Loop?

A for loop is used to iterate (repeat) over a sequence like:

- List
- Tuple
- String
- Dictionary
- Set
- Range()

It executes a block of code for each item in the sequence.

Basic Syntax

for variable in sequence:

code block

1 Looping Through a List

fruits = ["apple", "banana", "cherry"]

for x in fruits:

print(x)

Output:
apple
banana
cherry

2 Looping Through a String

for letter in "Mithun":

print(letter)

Each character prints separately.

3 Using range()

range() is mostly used in for loops.

Syntax:

range(start, stop, step)

- start -> starting value
- stop -> stops before this value
- step -> increment value

Example 1

for i in range(5):

print(i)

Output:

```
0
1
2
3
4
```

Example 2

for i in range(1, 6):

print(i)

Output:

```
1
2
3
4
5
```

Example 3 (Step)

for i in range(0, 10, 2):

print(i)

Output:

```
0
2
4
6
8
```

4 Looping Through Dictionary

```
student = {
  "name": "Mithun",
  "age": 23,
  "course": "MBA"
}
```

for key in student:
print(key)
Prints only keys.

Print Values

for value in student.values():
print(value)

Print Keys and Values

for key, value in student.items():
print(key, value)

- used to check whether an item exists in a set
- keyword used: in
- returns True or False
- not in is used to check absence
- membership checking in sets is fast

Syntax:

item in setname
item not in setname

Example:

```
a = ("apple", "banana", "cherry")
print("banana" in a)
print("grape" not in a)
```

Frozenset na immutable version of set change panna mudiyadhu and down is syntax

```
x = frozenset(("apple", "banana", "cherry"))
print(x)
print(type(x))
```

and we can use the below commands for frozenset

Method	Shortcut	Description	Try it
<code>copy()</code>		Returns a shallow copy	Try it >
<code>difference()</code>		Returns a new frozenset with the difference	Try it >
<code>intersection()</code>		Returns a new frozenset with the intersection	Try it >
<code>isdisjoint()</code>		Returns whether two frozensets have an intersection	Try it >
<code>issubset()</code>	$\subseteq$ / $\subset$	Returns True if this frozenset is a (proper) subset of another	Try it >
<code>issuperset()</code>	$\supseteq$ / $\supset$	Returns True if this frozenset is a (proper) superset of another	Try it >
<code>symmetric_difference()</code>		Returns a new frozenset with the symmetric differences	Try it >
<code>union()</code>	$\cup$	Returns a new frozenset containing the union	Try it >

Python Range:

Python la range oru sequence of number or main ah specific number of items repeat pandrathu

This set of numbers has its own data type called range.

main syntax is: **range(start,stop,step)**

common ah range print panna..**x=range(10)**
ipdi kudutha like a single it will consider 10 as stop number and 0 as start number
it will print from 0 to 9

or namma (5,10) kudutha 5 to 9 print aagum like the last number will not get printed...

three number kuduthaa first
indexnumber,lastnumber,number of steps to increase...

and most important of all oru range display we have to convert that to a list...

or like print(*variablenameofrange)

Python Arrays

Array unaku teriyum la da...

like storing value inside a variable...like cars=["Ford", "Volvo", "BMW"]

looping um easy daan for x in cars: print (x)

adding a element in array ku .append()

remove ku .pop() for random element... .pop(2) for a specific element..or remove ("elementtoberemoved")

and below are the some of important methods of array...

Method	Description
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the first item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

vera onnum perusa illa...same as other data collection methods..

Python JSON — Complete Notes

What is JSON?

JSON = JavaScript Object Notation

```
print("Friday")
case 6:
  print("Saturday")
case 7:
  print("Sunday")
```

Default value is case _: underscore is very important

Values combine panalam using or statement |

```
day = 4
match day:
  case 1 | 2 | 3 | 4 | 5:
    print("Today is a weekday")
  case 6 | 7:
    print("I love weekends!")
```

If statement ulla thinikalam:

```
month = 5
day = 4
match day:
  case 1 | 2 | 3 | 4 | 5 if month == 4:
    print("A weekday in April")
  case 1 | 2 | 3 | 4 | 5 if month == 5:
    print("A weekday in May")
  case _:
    print("No match")
```

Namma idea dhan ellam...!

Python Functions:

A function is a block of code which only runs when it called

A function can return data as a result

A function helps avoiding code repetition

How to create a function:

```
def function_name():
    print("Hi this is Mithun")
```

calling a function:

function_name()

a function can be called many times...

name can be separated by _ or we can use camel cases

```
def fahrenheit_to_celsius(fahrenheit):
    return (fahrenheit - 32) * 5 / 9

print(fahrenheit_to_celsius(77))
print(fahrenheit_to_celsius(95))
print(fahrenheit_to_celsius(30))
```

main ah concept idhu dhaan...ok va arguments...

Arguments:

function call pandrapo bracket la kudukura thing name is argument...

Parameter:

function laiy bracket kudukuradhu name dhan parameters

Positioning arguments:

while calling three or four arguments...we should match the order...ok va...lets see an example

```
def functionname(para1,para2,para3):
    printsomething
    printsomething
functionname(arg1,arg2,arg3)
```

***args -**

```
def my_function(*kids):
    print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

What is "args"?

The "args" parameter allows a function to accept any number of positional arguments. Inside the function, args becomes a tuple containing all the passed arguments.

Example

Accepting individual arguments from "args":

```
def my_function(*args):
    print("Type:", type(args))
    print("First argument:", args[0])
    print("Second argument:", args[1])
    print("All arguments:", args)

my_function("Emil", "Tobias", "Linus")
```

Scope:

A variable is only available from inside the region is created.This is called **scope**.

Local Scope:

function kulla irukura variable is local scope and can be accessed within the function

we can give function inside function...

Global Scope:

variable created in the main body of the python...idha namma endha function la vena access panikalam andha maari oru nalla scope.

```
def outer():
    def inner():
        print("inside inner")
    inner()
```

outer()

***args and **kwargs:**

By default, a function must be called with the correct

```
pop() Removes the element with the specified key
popitem() Removes the last inserted key value pair
setdefault() Returns the value of the specified key. If the key does not exist, insert the key, with the specified value
update() Updates the dictionary with the specified key value pairs
values() Returns a list of all the values in the dictionary
```

Python Iterators:

Iterator என்றால் என்ன?
ஒரு தொகுதி வரை values யும் தராமல், ஒவ்வொரு பட்சமையும் ஒரு value மட்டும் தருவது தான் Iterator

athavathu onnu illa ipo oru channel plus pandrom ipo suntv -> vijaytv trmba namma +click panna it wont stay on vijay tv next vijay tv hd ku pogum...ok va that is iterator..ok va lets see the syntax...

```
mytuple = ("apple", "banana", "cherry")
myit = iter(mytuple)

print(next(myit))
print(next(myit))
print(next(myit))
```

ipo single word kudutha it will take character as index aana 2 or more words kudutha it will take word as index...

first covert tuple into iter then access that iter with next method..

for looping maari dhaan it also remembers the current index position...so

Python Modules:

Modules in python are like and same as code library...like namma oru sila functions a oru thani python file la eludhi vachukalam aprm import method use pani andha file la irukura functions use panikalam

Steps:

Create a Module
To create a module just save the code you want in a file with the file extension .py

Example
Save this code in a file named mymodule.py

```
def greeting(name):
    print("Hello, " + name)
```

Use a Module
Now we can use the module we just created, by using the `import` statement.

Example
Import the module named mymodule, and call the greeting function:

```
import mymodule

mymodule.greeting("Tobias")
```

same namma variables um access panikalam another python file...

Naming a module is simple as pythonfile.py

renaming is like

Create an alias for mymodule called mx:

```
import mymodule as mx

a = mx.person1["age"]
print(a)
```

built in modules um neriya iruku like platform...

main aana concept idhu dhan...

Python Math:

Python has a set of built-in math functions, including an extensive math module, that allows you to perform mathematical tasks on numbers.

x= min() idhukulla iruka elements la minimum value
y=max() idhukulla iruka elements la maximum value

abs(use panna oppostive number kedaikum like pos to neg or neg to pos)

pow(number1,number2) - number1 power of number2

math oru builtin module...import use pani import
pnadikalam namma program la

Python Math Functions — Common Methods Table

Function	Usage	Meaning (Simple)	Example	Output
<code>min()</code>	Smallest value	Siriyana number	<code>min(5,10,25)</code>	5
<code>max()</code>	Largest value	Periya number	<code>max(5,10,25)</code>	25
<code>abs()</code>	Absolute value	Negative → Positive	<code>abs(-7)</code>	7
<code>pow(a,b)</code>	Power	a^b	<code>pow(2,3)</code>	8
<code>round()</code>	Round number	Nearby integer	<code>round(4.6)</code>	5
<code>math.sqrt(x)</code>	Square root	$\sqrt{x}$	<code>math.sqrt(25)</code>	5.0
<code>math.ceil(x)</code>	Round up	Mel integer	<code>math.ceil(4.5)</code>	5

Print Values

for value in student.values():
print(value)

Print Keys and Values

for key, value in student.items():
print(key, value)

break Statement

Stops the loop immediately.
for i in range(10):
if i == 5:
break
print(i)
Stops at 5.

continue Statement

Skips current iteration.
for i in range(5):
if i == 2:
continue
print(i)
2 will not print.

else in For Loop

Runs when loop finishes normally.
for i in range(3):
print(i)
else:
print("Loop finished")

Nested For Loop

Loop inside another loop.
for i in range(3):
for j in range(2):
print(i,j)

Loop with Index (Important)

Use enumerate()
fruits = ["apple", "banana", "cherry"]
for index, value in enumerate(fruits):
print(index, value)

Practical Example

Multiplication Table
num = 5
for i in range(1,11):
print(num, "x", i, "=", num * i)

Important Points to Remember

- ✓ For loop works on sequences
- ✓ range() stops before the end value
- ✓ Indentation is compulsory
- ✓ break → stop
- ✓ continue → skip
- ✓ enumerate() → index + value

if you want, next I'll give:

-  10 practice tasks
- or  Mini test
- or  Real interview-style problems

Sollu da 

Python Dates:

to access date and time from a built in python library....

oru module import pananum for this..the name is datetime...

import datetime

verum date fetch panna oru sample program...

import datetime

x=datetime.datetime.now()

print(x.year) will give current year (2026)

Directive	Description	Example
%a	Weekday, short version	Wed
%A	Weekday, full version	Wednesday
%w	Weekday as a number 0-6, 0 is Sunday	3
%d	Day of month 01-31	31
%b	Month name, short version	Dec
%B	Month name, full version	December
%m	Month as a number 01-12	12
%y	Year, short version, without century	18
%Y	Year, full version	2018
%H	Hour 00-23	17
%I	Hour 00-12	05
%p	AM/PM	PM
%M	Minute 00-59	41
%S	Second 00-59	08
%f	Microsecond 000000-999999	548513
%z	UTC offset	+0100
%Z	Timezone	CST
%j	Day number of year 001-366	365
%U	Week number of year, Sunday as the first day of week, 00-53	52
%W	Week number of year, Monday as the first day of week, 00-53	52
%c	Local version of date and time	Mon Dec 31 17:41:00 2018
%C	Century	20
%x	Local version of date	12/31/18
%X	Local version of time	17:41:00
%%	A % character	%
%G	ISO 8601 year	2018
%u	ISO 8601 weekday (1-7)	1
%V	ISO 8601 weeknumber (01-53)	01

Python Pip (TERMINAL PACKAGE):

pip install packagename - to install a package

What is JSON?

JSON = JavaScript Object Notation

JSON is a format used to store and exchange data.

It is text, written using JavaScript object style.

Mostly used for:

- APIs
- Web applications
- Data transfer between frontend & backend
- Configuration files

Simple Meaning

JSON = Python dictionary structure but stored as TEXT format

Import JSON Module

```
import json
```

Python is case-sensitive → json (not JSON)

Sample JSON

```
{
  "name": "Mithun",
  "age": 23,
  "isStudent": true
}
```

Python vs JSON Difference

Python	JSON
dict	object
True	true
False	false
None	null

' or " allowed only " allowed

Python object Text format

JSON Functions Overview

loads / dumps → STRING

load / dump → FILE

1. json.dumps() — Python → JSON String
Convert Python object into JSON string.

```
import json
```

```
data = {"name": "Mithun", "age": 23}
```

```
json_data = json.dumps(data)
```

```
print(json_data)
print(type(json_data))
Output
```

```
{"name": "Mithun", "age": 23}
```

```
<class 'str'>
```

✓ dumps = dump string

2. json.loads() — JSON String → Python
Convert JSON string into Python dictionary.

```
import json
```

```
x = '{"name": "Mithun", "age": 23}'
```

```
data = json.loads(x)
```

```
print(data["name"])
print(type(data))
Output
```

```
Mithun
```

```
<class 'dict'>
```

✓ loads = load string

3. json.dump() — Write JSON to File
Write Python object into a JSON file.

```
import json
```

```
data = {"name": "Mithun", "age": 23}
```

with open("data.json", "w") as f:
 json.dump(data, f)

✓ Creates data.json

4. json.load() — Read JSON from File
Read JSON file and convert into Python object.

```
import json
```

with open("data.json", "r") as f:
 data = json.load(f)

```
print(data)
```

Important Concept (VERY IMPORTANT)

```
outer()
```

*args and **kwargs:

By default, a function must be called with the correct number of arguments..

However, sometimes you may not know how many arguments that will be passed into your function.

*args and **kwargs allow functions to accept a unknown number of arguments.

Arbitrary arguments - *args:

namaku ethana input varum nu teriyadhu so idha
namma use pandrom same for **kwargs

tuple la dha use panna mudiyhu

If you do not know how many arguments will be passed into your function, add a * before the parameter name. This way, the function will receive a tuple of arguments and can access the items accordingly:

```
Using *args to accept any number of arguments:

def my_function(*kids):
    print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

Arbitrary keyword arguments - **kwargs:

If you do not know how many keyword arguments will be passed into your function, add two asterisks ** before the parameter name. This way, the function will receive a dictionary of arguments and can access the items accordingly:

Example

Using **kwargs to accept any number of keyword arguments:

```
def my_function(**kid):
    print("His last name is " + kid["lname"])

my_function(lname = "Tobias", lname = "Refsnes")
```

comma venam na sep=","

Python Try Except:

try - oru block of code try pannu da

except - run aagalaina indha block a run pannu

```
try:
    num = int(input("Enter number: "))
    print(10 / num)
except:
    print("Error occurred!")
```

ipdi namma print podalam or if we know the type of error that would occur andha error a direct ah podalam like below screenshot..

```
try:
    x = int(input("Enter number: "))
    print(10 / x)

except ValueError:
    print("Enter numbers only")

except ZeroDivisionError:
    print("Zero allowed illa")
```

else- error illa na try mudichutu else part
run aagum irundha error ku poirum

```
try:
    x = int(input())
except ValueError:
    print("Wrong input")
else:
    print("Success:", x)
```

finally - error iru illama po aana na run
aaven nu soldra oru aalu..

```
try:
    print("Try block")
except:
    print("Error")
finally:
    print("Always runs")
```

chatgpt sonnadh:

TRY → Bike ride start pannura

EXCEPT → Slip aana first aid

ELSE → Safe ah destination reach

FINALLY → Bike lock pannitu veetuku po

Full Structure

round()	Round number	Nearby integer	round(4.6)	5
math.sqrt(x)	Square root	√x	math.sqrt(25)	5.0
math.ceil(x)	Round up	Mel integer	math.ceil(4.2)	5
math.floor(x)	Round down	Keela integer	math.floor(4.9)	4
math.factorial(x)	Factorial	x!	math.factorial(5)	120
math.pi	Pi value	π constant	math.pi	3.14159
math.log(x)	Log value	Natural log	math.log(10)	2.30

Python Random:

Function	Usage	Meaning (Simple)	Example	Sample Output
random.random()	Random float	0-1 between number	random.random()	0.53
random.randint(a,b)	Random integer	a to b number	random.randint(1,10)	7
random.randrange()	Random from range	Step range number	random.randrange(0,10,2)	4
random.choice()	Random item	List la oru item	random.choice(['a','b','c'])	'b'
random.shuffle()	Mix list	List order change	random.shuffle(list)	shuffled list
random.uniform(a,b)	Random decimal	Float between range	random.uniform(1,5)	3.42
random.sample()	Random selection	Multiple unique items	random.sample([1,2,3,4],2)	[2,4]

Python OS:

OS Module (System / Files) ★★Backend Important

os module — Small Definition
os module in Python is used to interact with the operating system.

Simple ah sollannu na:

Python-la namma computer files, folders, paths, system operations control panna use pannradhu os module.

Easy Tamil Meaning

- Folder create panna
 - File delete panna
 - Current directory check panna
 - File iruka illaya check panna
- ithu ellam os module use panni pannalam.

```
import os
```

Function	Why Used Mostly	Example
os.getcwd()	current folder	os.getcwd()
os.listdir()	list files	os.listdir()
os.mkdir()	create folder	os.mkdir("test")
os.remove()	delete file	os.remove("a.txt")
os.path.exists()	check file	os.path.exists("a.txt")

✓ Used in: file handling, automation scripts, backend.

STRING FORMATTING

f-string is used to insert variables or values directly inside a string without writing separate print statements.

Example:

```
txt = f"The price is 49 dollars"
print(txt)
```

Taking input and using f-string:

```
name = input("ENTER YOUR NAME: ")
print(f"Hi {name}.. Hearty Welcome")
```

We can also use numbers (int or float) inside f-strings.

Example:

```
price = 59
txt = f"The price is {price:.2f} dollars"
print(txt)
```

Here:

:.2f → shows the number with 2 decimal places.

Inside {} brackets we can:

- use variables
- perform mathematical operations
- use string functions

Examples:

```
a = 10
b = 5
print(f"Sum is {a + b}")
```

```
name = "mithun"
print(f"Uppercase name: {name.upper()}")
```

Important:

The {} inside an f-string evaluates expressions dynamically and inserts the result into the string.

Python Pip (TERMINAL PACKAGE):

pip install packagename - to install a package

pip uninstall packagename - to uninstall a package

pip show packagename - to display the details of a package (version, location, dependencies)

pip download packagename - just downloading without installing

pip freeze - freeze all packages

pip install --upgrade packagename - to upgrade a package..

pip list - list all the packages..

pip check - shows broken dependencies

pip cache dir - show cache

pip cache purge - remove cache

pip install . - install this folder as a Python package.

Python RegEx (Regular Expression):

A RegEx or Regular Expression is a sequence of characters that forms a search pattern.

RegEx is used to:

- check if a string contains a specific pattern
- search text
- extract matching data
- replace text
- validate input (email, phone number, password, etc.)

Importing RegEx in Python:

To use RegEx, first import the module:

```
import re
```

Common RegEx Functions:

- re.search(pattern, string)
- Searches for a pattern anywhere in the string
- Returns match object if found
- re.findall(pattern, string)
- Returns all matches as a list
- re.sub(pattern, replacement, string)
- Replaces matched text with another string
- re.split(pattern, string)
- Splits string using a pattern

Important Syntax (Patterns):

```
\d -> digit (0-9)
\D -> non-digit
\w -> word character (a-z, A-Z, 0-9, _)
\W -> non-word character
\s -> whitespace (space, tab)
\S -> non-whitespace
. -> any character
```

Quantity Symbols:

```
+ -> one or more times
* -> zero or more times
? -> optional (0 or 1 time)
{n} -> exactly n times
{n,m} -> between n and m times
```

Example:

```
\d+ -> one or more digits
\d{10} -> exactly 10 digits
```

Position Anchors:

```
^ -> start of string
$ -> end of string
```

Example:

```
^d{10}$ -> string must contain exactly 10 digits
```

Character Sets:

```
[a-z] -> lowercase letters
[A-Z] -> uppercase letters
[a-zA-Z] -> alphabets only
[0-9] -> digits
[^0-9] -> anything except digits
```

Special Operators:

```
| -> OR operator
() -> grouping
```

```
data = json.load(f)
```

```
print(data)
```

🧠 Important Concept (VERY IMPORTANT)
JSON is always TEXT
Example:

```
x = {'city': "Madurai", "temp": 34}'  
Before loads():
```

```
type → str  
After loads():
```

```
type → dict
```

⚡ Access JSON Data

```
print(data["temp"])  
Output:
```

```
34  
JSON → Python dict → access using keys.
```

★ Real-World Usage
JSON is used in:
☒ REST APIs
☒ Django / Flask backend
☒ Mobile apps
☒ Web servers
☒ Database communication
Example API response:

```
{  
  "weather": "sunny",  
  "temp": 32  
}
```

Python:

```
data["temp"]
```

🧠 Memory Trick

```
s = string  
(no s) = file  
Function Use  
dumps() Python → JSON string  
loads() JSON string → Python  
dump() Python → JSON file  
load() JSON file → Python
```

! Common Mistakes
✗ Using single quotes in JSON

```
{'name': 'Mithun'} # WRONG JSON  
☒ Correct JSON
```

```
{"name": "Mithun"}
```

🔧 Mini Practice

```
import json
```

```
x = {'city': "Madurai", "temp": 34, "weather":  
"sunny"}
```

```
data = json.loads(x)
```

```
print(data["weather"])  
Expected Output:
```

```
sunny
```

☒ One-Line Summary

JSON = Data exchange format
Python uses json module to convert
between dictionary ↔ text ↔ file.

Python	JSON
dict	Object
list	Array
tuple	Array
str	String
int	Number
float	Number
True	true
False	false
None	null



💡 One-Line Memory Trick:

🔧 TRY = attempt
🔧 EXCEPT = handle problem
🔧 ELSE = success case
🔧 FINALLY = cleanup

SAMPLE CODE:

```
balance = 50000000
```

```
try:  
    print("===== ATM MACHINE =====")  
  
    amount = int(input("Enter withdrawal  
amount: "))  
  
    if amount <= 0:  
        raise ValueError("Amount must be  
positive")
```

```
    if amount > balance:  
        raise Exception("Insufficient  
balance")
```

```
except ValueError as e:  
    print("Invalid input:", e)
```

```
except Exception as e:  
    print("Transaction failed:", e)
```

```
else:  
    balance -= amount  
    print("Withdrawal successful ☒)  
    print("Remaining balance:", balance)
```

```
finally:  
    print("Transaction completed. Thank  
you for using ATM 🙏")
```

TRUTHY AND FALSY VALUES

Definition

In Python, every value is treated as either True or False when used inside conditions like if statements.

Python automatically decides whether a value is True (Truthy) or False (Falsy).

Falsy Values (Automatically False)

The following values are considered False:

```
False  
None  
0  
0.0  
"" (empty string)  
[] (empty list)  
{ } (empty dictionary)  
set() (empty set)
```

Example:

```
if "":  
    print("True")  
else:  
    print("False")
```

```
Output:  
False
```

Truthy Values (Automatically True)

Any value that is NOT empty or zero is considered True.

Examples:

```
"hello"  
10
```

```
print(f"Uppercase name: {name.upper()}")
```

Important:

The {} inside an f-string evaluates expressions dynamically and inserts the result into the string.

NONE

Definition

None is a special value in Python that represents the absence of a value.
It is not 0, not False, and not an empty string.
None simply means "no value" or "nothing assigned".

Basic Example

```
x = None  
print(x)
```

```
Output:  
None
```

Checking None

We usually check None using the "is" keyword.

```
x = None
```

```
if x is None:  
    print("No value present")
```

Important Note

```
Use:  
is None
```

```
Avoid:  
== None
```

Because "is" checks identity and is the recommended Python practice.

Common Use Cases

- Default function values
- When a variable has no data yet
- Functions that do not return anything explicitly return None

Example with Function

```
def greet():  
    print("Hello")
```

```
result = greet()  
print(result)
```

```
Output:  
Hello  
None
```

USER INPUT

Definition

User input allows a program to take data from the user during runtime.
In Python, input() is used to receive input from the keyboard.

Basic Example

```
name = input("Enter your name: ")  
print(name)
```

Note

input() always returns data as a string.

Input as Integer

```
age = int(input("Enter your age: "))  
print(age)
```

Input as Float

```
price = float(input("Enter price: "))  
print(price)
```

Using Input with f-string

```
name = input("Enter your name: ")  
print(f"Welcome {name}")
```

Multiple Inputs

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
print(f"Sum is {a + b}")
```

Common Mistake

```
num = input("Enter number: ")  
print(num + 5)
```

This causes an error because input is a string.

Correct Way

```
num = int(input("Enter number: "))  
print(num + 5)
```

Special Operators:

```
| -> OR operator  
() -> grouping
```

Example:
cat|dog -> matches "cat" or "dog"

Raw String (Important in Python):

Always use raw string for regex patterns:

```
pattern = r"\d+"
```

Common Examples:

Find numbers:
r"\d+"

Email pattern (basic):
r"\w+@\w+\.\w+"

Date format:
r"\d{2}-\d{2}-\d{4}"

Phone number:
r"^\d{10}\$"

Summary:

RegEx = Pattern matching tool used for searching, validating, extracting, and replacing text efficiently.

Python RegEx – Step by Step Workflow

Step 1: Import the RegEx module

```
import re
```

Step 2: Create a string (data to search)

```
text = "My phone number is 9876543210"
```

Step 3: Write a pattern (what you want to find)

```
pattern = r"\d+"
```

Step 4: Choose what action you want

A) Search (check if pattern exists)

```
result = re.search(pattern, text)  
print(result)
```

B) Find all matches

```
result = re.findall(pattern, text)  
print(result)
```

C) Replace matches

```
result = re.sub(pattern, "X", text)  
print(result)
```

D) Split using pattern

```
result = re.split(pattern, text)  
print(result)
```

Step 5: Print or use the result

```
print(result)
```

Simple Mental Flow:

```
import re  
↓  
string create  
↓  
pattern create  
↓  
choose function (search/findall/sub/split)  
↓  
print result
```

```
3.5
[1,2]
{"name":"Mithun"}
```

Example:

```
if "hello":
    print("Runs because it is True")
```

Output:
Runs because it is True

Why This Is Useful

We can directly check values without writing comparisons.

Example:

```
name = input("Enter name: ")

if name:
    print("Name entered")
else:
    print("Empty input")
```

No need for:

```
if name != ""
```

Real World Usage

Used in:

- form validation
- login systems
- checking API responses
- checking empty data

Memory Trick

Empty → False
Has value → True